

Submission Worksheet

Submission Data

Course: IT202-008-S2026

Assignment: IT202 Milestone 2

Student: Emperor G. (eg278)

Status: Submitted | **Worksheet Progress:** 100%

Potential Grade: 802.00/802.00 (100.00%)

Received Grade: 0.00/802.00 (0.00%)

Started: 4/19/2026 3:36:46 PM

Updated: 5/2/2026 6:24:00 PM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT202-008-S2026/it202-milestone-2/grading/eq278>

View Link: <https://learn.ethereallab.app/assignment/v3/IT202-008-S2026/it202-milestone-2/view/eq278>

Instructions

- Overview Link: https://youtu.be/t6X8dzZt_a0
- 1. Refer to Milestone2 of this doc:
<https://docs.google.com/document/d/1XE96a8DQ52Vp49XACBDTNCq0xYDt3kF29cO88EWVwfo/view>
- 2. Ensure you read all instructions and objectives before starting.
- 3. Ensure you've gone through each lesson related to this Milestone
- 4. Switch to the Milestone2 branch
 - 1. `git checkout Milestone2` (ensure proper starting branch)
 - 2. `git pull origin Milestone2` (ensure history is up to date)
- 5. Fill out the below worksheet
 - Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
 - Ensure proper styling is applied to each page
 - Ensure there are no visible technical errors; only user-friendly messages are allowed
- 6. Once finished, click "Submit and Export"
- 7. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
 - 1. `git add .`
 - 2. `git commit -m "adding PDF"`
 - 3. `git push origin Milestone2`
 - 4. On Github merge the pull request from Milestone2 to qa
 - 5. On Github create a pull request from qa to prod and immediately merge. (This will trigger the prod deploy to make the Render.com prod links work)
- 8. Upload the same PDF to Canvas
- 9. Sync Local
 - 1. `git checkout qa`
 - 2. `git pull origin qa`

Section #1: (2 pts.) Data

Progress: 100%

Task #1 (0.50 pts) API Data Table

Part 1:

Progress: 100%

Details:

- Show the table(s) structure of the table made to hold your API
- There should be at least 3 properties/fields beyond id, created, modified, and respective is_api or api_id columns
- Logical types and constraints should be set
- Show at least a few records from each API table (samples should include both API fetched data and custom entries noted by an api_id or is_api column)

File	Status	Details
001_create_table_users.sql (Show SQL)	✓	Table already exists (skipped)
002_alter_table_users.sql (Show SQL)	⚠	Warning (duplicate)
003_create_table_roles.sql (Show SQL)	✓	Table already exists (skipped)
004_create_table_tournaments.sql (Show SQL)	✓	Table already exists (skipped)
005_insert_admin_role.sql (Show SQL)	✓	Success
006_create_table_golf.sql (Show SQL)	✓	Success
<pre>CREATE TABLE "ET202-M25-Golf" ("id" int NOT NULL AUTO_INCREMENT PRIMARY KEY, "tourn_id" varchar(50) NOT NULL, "name" varchar(255) NOT NULL, "start_date" date NOT NULL, "end_date" date NOT NULL, "created" timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP, "modified" timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, "is_api" tinyint(1) DEFAULT '1');</pre>		Success array (0 => 'success', 1 => NULL, 2 => NULL,)

properties/field

id	tourn_id	name	start_date	end_date	created	modified
1	016	The Sentry	2024-01-06	2024-01-09	2026-04-20 15:03:55	2026-04-20 15:03:55
2	017	Sony	2024-01-12	2024-01-14	2026-04-20 15:03:55	2026-04-20 15:03:55
3	CUSTOM1	Today Custom Tournament	2024-06-02	2024-06-04	2026-04-20 15:03:56	2026-04-20 15:03:56
4	ETGolf	Golf	2024-01-12	2024-01-17	2026-04-23 22:55:14	2026-04-23 22:55:14
5	Money	Wrestlemania	2024-01-10	2024-01-15	2026-04-23 23:55:51	2026-04-23 23:55:51

API Sample Records

 Saved: 5/2/2026 6:24:00 PM

Part 2:

Progress: 100%

Details:

- Add a direct link to your sql file from Github (should end in .sql)

URL #1

<https://github.com/eq278-cmd/eq278->


URL

<https://github.com/eq278-cmd/eq278->



Saved: 5/2/2026 6:24:00 PM

Part 3:

Progress: 100%

Details:

- Briefly note what each field/property represents

Your Response:

So in the sql table, firstly, id represents a unique identifier for each golf tournament record. Secondly, tourn_id represents the tournament's unique id from the external API; this will connect the user's stored data to the API's data. The name field is basically the name of the golf tournament. Start_date represents the date when the tournament begins. End-date is the date when the tournament ends. Created represents the timestamp displaying when the record was initially inserted into the database. Modified represents the timestamp that updates automatically whenever the record changes. Lastly, is_api determines whether the record originated from the API(1 =yes); it's meant to remove API from manually inserted data.



Saved: 5/2/2026 6:24:00 PM

Task #2 (0.50 pts.) - Data Transformation

Progress: 100%

Part 1:

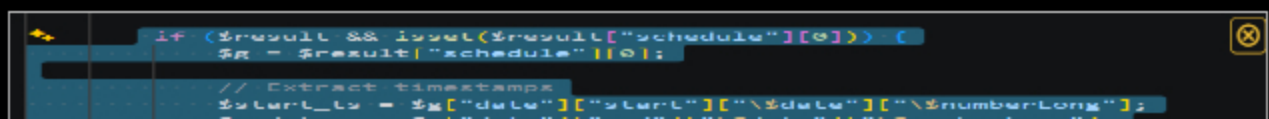
Progress: 100%

Details:

- Show the code that handles the data transformation/conversion (from the API)
- Show a Render.com qa example of API getting fetched and loaded into the DB



render example



```
// Transform into DB-ready fields
$quote = [
  "tourn_id" => $q["tournId"],
  "name" => $q["name"],
  "start_date" => date("Y-m-d", $start_ts / 1000),
  "end_date" => date("Y-m-d", $end_ts / 1000),
  "is_api" => 1
];
} else {
  flash("API returned no results", "warning");
  You, 2 days ago - update create golf
}
```

transformation code

 Saved: 4/29/2026 12:27:39 PM

Part 2:

Progress: 100%

Details:

- Include Render.com prod link to API fetch page
- Include pull request link for this feature

URL #1

https://eg278-it202-008-prod.onrender.com/project/admin/create_golf.php



URL

https://eg278-it202-008-prod.onrender.com/project/admin/create_golf.php



URL #2

<https://github.com/eg278-cmd/eg278-it202-008-pullrequest>



URL

<https://github.com/eg278-cmd/eg278-it202-008-pullrequest>



 Saved: 4/29/2026 12:27:39 PM

Part 3:

Progress: 100%

Details:

- Will you be using all the data or just a subset?
- Do you need to convert any data or extract pieces of a property's value?
- Briefly explain the transformation/extraction logic to get the API data to the shape your application intends to use

Your Response:

- I will only using a subset of my api data. The result will show an organized schedule format that involves certain properties like tourn_id, name, date.start.date.\$numberLong, and date.end.date.\$numberLong
- I need to convert any data/extract pieces because my golf api has timestamps converted to milliseconds. I will need to extract \$numberLong value and convert it from milliseconds to seconds in order for the date() can have it in a accurate format. Transformation Logic:
 1. Once the API is called, schedule[0] will get the first tournament schedule
 2. The timestamp values(start_is = \$g..., \$end_ts = \$g...) will be extracted
 3. convert milliseconds to seconds by 1000

4. Get the timestamps into the Y-m-d format using date()
5. Form a \$quote array with the main database fields
6. the transformed data is inserted into the golf table



Saved: 4/29/2026 12:27:39 PM

⇒ Task #3 (0.50 pts.) - API Duplicates

Progress: 100%

Details:

- Consider how duplicate entries are handled
 - What happens if you get the same result from the API for an entity that exists but has been unmodified?
 - What happens if you get the same result from the API for an entity that exists but has been modified?

Your Response:

So during this process, my database contains a UNIQUE constraint on tourn_id which implies that if the API returns a previous existed tournament in the table, MySQL will immediately block that insert and i'll receive an error message in result of a duplicate row.

- If not modified and the API returns the same tournament again, then the MySQL will trigger a duplicate key error and the PHP code retrieves that error and shows "An error occurred". There will be no new rows just the existing row.
- If modified and the API returns a tournament with the same tourn_id except with a different data, the insert will still fail and the existing row won't be updated.



Saved: 4/29/2026 4:51:59 PM

⇒ Task #4 (0.50 pts.) - API Fetching

Progress: 100%

Details:

- Consider how your application will trigger this data fetch
 - (i.e., periodic via cron job, poor man's cron job, on app start/awake, etc)
 - (i.e., User or Admin triggered via a specific page/action)
 - (i.e., Lazy loaded during searches for stale/missing content)

Your Response:

Within my application, the API fetch is effected by a specific Admin through a

• Within my application, the API fetch is affected by a specific Admin through a page. The admin page is /project/admin/create_golf.php. Once the user clicks the Fetch API button, this will send a POST request that calls the fetch_golf_schedule and the next tournament is inserted. This is very important for the project because it gives the admin control over whenever there is new data being added. However, my app don't involve any cron job, poor man's cron, lazy loading, nor on app start. I mean, these features can be applied in the future but this one an admin triggered fetch.



Saved: 4/29/2026 5:06:23 PM

Section #2: (150 pts.) Data Creation

Progress: 100%

≡ Task #1 (60 pts.) - Validations

Progress: 100%

📁 Part 1:

Progress: 100%

Details:

- Show the html validations (code)
- Show the javascript validations (code)
- Show the php validations (code)

```
<form method="POST">
  <div class="mb-3">
    <label>Tournament ID</label>
    <input type="number" name="tourn_id" required>
  </div>
  <div class="mb-3">
    <label>Tournament Name</label>
    <input type="text" name="name" required>
  </div>
  <div class="mb-3">
    <label>Start Date</label>
    <input type="date" name="start_date" required>
  </div>
  <div class="mb-3">
    <label>End Date</label>
    <input type="date" name="end_date" required>
  </div>
</form>
```

HTML Validations(code)

```
if ($result && isset($result["schedule"][0])) {
    $g = $result["schedule"][0];

    // Extract timestamps
    $start_ts = $g["date"]["start"]["\${date}"]["\${numberLong}"];
    $end_ts = $g["date"]["end"]["\${date}"]["\${numberLong}"];

    // Transform into DB-ready fields
    $quote = [
        "tourn_id" => $g["tournid"],
        "name" => $g["name"],
        "start_date" => date("Y-m-d", $start_ts / 1000),
        "end_date" => date("Y-m-d", $end_ts / 1000),
        "is_api" => 1
    ];
    $sql = "#36-42 \$quote =";
    $sql = "INSERT INTO tournaments ($quote) VALUES ($quote)";
    $flash("API returned no results", "warning");
}
```

PHP Validations(code) (API Validation)


```
❖ foreach ($_POST as $k => $v) {  
    ..... if (!in_array($k, ["tourn_id", "name", "start_date", "end_date"])) {  
    ..... unset($_POST[$k]);  
    ..... }  
    ..... }  
    ..... }  
    You, 5 days ago • updated transformation code <- #49-53 foreach ($
```

PHP Validations(code) (POST sanitization Validation)

```
❖ try {  
    ..... $stmt = $db->prepare($query);  
    ..... $stmt->execute($params);  
    ..... flash("Inserted record " . $db->lastInsertId(), "success");  
    ..... } catch (PDOException $e) {  
    ..... error_log("DB Error: " . var_export($e, true));  
    ..... flash("An error occurred", "danger");  
    ..... }  
    You, 20 hours ago • error changed
```

PHP Validations(code) (DB Validation)

```
<script>  
❖ function validate(form) {  
    ..... // TODO i: implement Javascript validation (you'll do this on your own to  
    ..... // ensure it returns false for an error and true for success  
    ..... let email = form.email.value.trim();  
    ..... let password = form.password.value.trim();  
    .....  
    ..... // Email and password can't be empty  
    ..... if (email.length === 0) {  
    .....     alert("Email or username is required.");  
    .....     return false;  
    ..... }  
    ..... if (password.length === 0) {  
    .....     alert("Password is required.");  
    .....     return false;  
    ..... }  
    ..... return true;  
    You, 4 weeks ago • added nav.php, updated register, f
```

JS Validation

 Saved: 5/1/2026 8:49:31 PM

≡ Part 2:

Progress: 100%

Details:

- Briefly explain the html validations used
- Briefly explain the javascript validation logic
- Briefly explain the php validation logic
- Note which role(s) have access to this create form

Your Response:

In my form, the HTML validation enables built-in attributes like "required" in which for each field in the "Create Tournament" such as tournament id, name, start date, and end date they have a required that prevents any empty submissions, while type=number and type=date make sure the user uses correct formatted values before the submission of the form. In regards to server side, PHP validation is used to ensure that the data is valid and to prevent SQL injection attacks. For example, the PHP code uses PDO to prepare and execute SQL queries, which helps prevent SQL injection. Additionally, the code uses error_log to log any database errors, and flash messages to provide feedback to the user.

PHP shows more validation to ensure safe data; once the form is submitted, PHP checks to see which action was affected, the sanitizes the POST data by getting rid of any identified fields, the fetch option will be there to verify that the API returned valid data. If any complications, PHP will use flash() to remind the user. It's important to know that access to the create form is unavailable, only the users with an Admin role have that access.



Saved: 5/1/2026 8:49:31 PM

Task #2 (60 pts.) - Examples

Progress: 100%

Details:

- Show a successful creation
- Show how a duplicate submission is handled (i.e., what message or behavior occurs)
- Demonstrate various validation errors
- Ensure Render.com qa url is visible



successful creation



Validation errors(HTML)



Validation errors(PHP)



Saved: 4/29/2026 7:26:18 PM

Task #3 (30 pts.) - Links

Progress: 100%

Details:

- Include the Render.com prod link to this page
- Include pull request link for this feature

URL #1

https://eg278-it202-008-prod.onrender.com/project/admin/create_golf.php#



URL

https://eg278-it202-008-prod.onrender.com/project/admin/create_golf.php#



URL #2

<https://github.com/eg278-cmd/eg278-it202-008-2026/pull/56>



URL

<https://github.com/eg278-cmd/eg278-it202-008-2026/pull/56>



Saved: 4/29/2026 8:10:34 PM

Section #3: (150 pts.) Data List Page (Many Items)

Progress: 100%

Task #1 (60 pts.) - Code

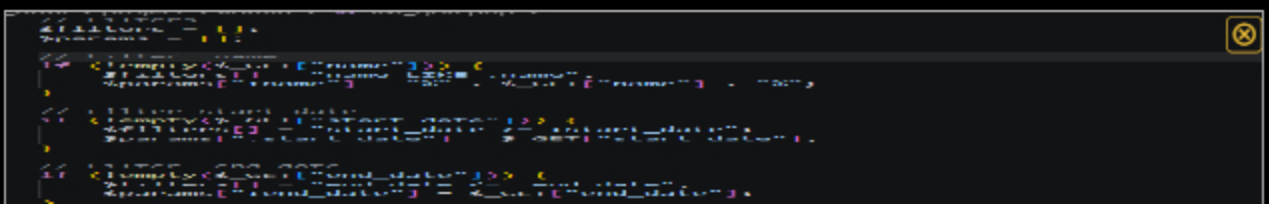
Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code that handles the filter/sort logic
- Show the code that enforces the record limit (valid range 1-100, default 10 if out of range)
- Show the code that generates the list/grid output
- Styling must be applied



```
// Limit
$limit = isset($_GET["limit"]) ? (int)$_GET["limit"] : 10;
if ($limit < 1 || $limit > 100) {
    $limit = 10;
}
```

filter/sort logic

```
// Limit
$limit = isset($_GET["limit"]) ? (int)$_GET["limit"] : 10;
if ($limit < 1 || $limit > 100) {
    $limit = 10;
}
```

record limit

```
$query = "SELECT id, name, id, name, start_date, end_date, is_api
FROM `tournaments`";

if (count($filters) > 0) {
    $query .= " WHERE " . implode(" AND ", $filters);
}

$query .= " ORDER BY $sort $order";
$query .= " LIMIT $limit";

$stmt = $db->prepare($query);

foreach ($params as $key => $value) {
    if ($key === "limit") {
        $stmt->bindValue($key, (int)$value, PDO::PARAM_INT);
    } else {
        $stmt->bindValue($key, $value);
    }
}

$stmt->execute();
$results = [];
try {
    $stmt->execute();
    $results = $stmt->fetchAll(PDO::FETCH_ASSOC);
}
```

list/grid output

```
<div class="golf-grid">
    <?php foreach ($results as $record) : ?>
        <div class="golf-card">
            <div><?php echo($record["name"]); ?></div>
            <div><?php echo($record["start_date"]); ?></div>
            <div><?php echo($record["end_date"]); ?></div>
            <div><?php echo($record["is_api"]); ?></div>
            <div><?php echo get_url("admin/edit_golf.php?id=" . $record["id"]); ?></div>
            <div><?php echo get_url("admin/delete_golf.php?id=" . $record["id"]); ?></div>
            <div><?php echo get_url("admin/view_golf.php?id=" . $record["id"]); ?></div>
        </div>
    </div>
</div>
```

styling

 Saved: 5/2/2026 5:41:59 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the filter/sort logic works and formulates the results
- Briefly explain how the limit is validated and applied server-side
- Briefly explain how the list/grid output is generated
- Briefly note the styling choices

Your Response:

- So the page will check on the inputs from the query that contains name, start_date, and end_date. Based on each provided filter input, an additional SQL condition will be inserted into the filters array and if it matches it'll be stored into \$params. The way sorting works is when the requested sort column and order is being validated. If any missing or invalid values, the system won't run successfully and start_date will be at default. When it involves the limit, there are restrictions where the minimum is 1, maximum is 100, but 10 would be the default. In this way, users won't be in a situation where to keep requesting huge amounts of result sets and checks if the query is safe and secure within the limit value. For each record, there's a container styled box that includes the tournament name, tournament id, start/end dates, api row indicator, and edit/view links; this forms a grid of the tournament cards for the filtered and sorted query outputs. Lastly, the CSS styling displays layout in different sizes: a light background, padding, rounded corners, subtle border.



Saved: 5/2/2026 5:41:59 PM

Task #2 (60 pts.) - Examples

Progress: 100%

Details:

- Show a few variations of filter/sort applied (including no records - should display a "no results" message)
- Show both API-fetched and manually created entities present in the list
- Show the limit field in use with at least two different values
- Each list item should be a summary
- Each list item should have the following links
 - A link to a single view of the specific entity (i.e., a details page)
 - A link to delete this entity (this may be an admin-only functionality, but it should be present for the respective role)
 - A link to edit this entity (this may be an admin-only functionality, but it should be present for the respective role)
- Ensure Render.com qa url is visible

List Golf Tournaments			
Golf Supremes ID: 444 Start: 2025-05-01 End: 2025-05-05 API Row: 0	TEST ID: 234 Start: 2024-07-01 End: 2024-07-05 API Row: 0	Wrestlemania ID: Money Start: 2024-01-10 End: 2024-01-15 API Row: 0	The Sentry ID: 016 Start: 2024-01-04 End: 2024-01-07 API Row: 1

no filters applied

← https://eg278-it202-008-2026-1-qa.onrender.com/project/admin/list_golf.php?name=Test

Home Profile Roles Golf Companies Logout

List Golf Tournaments

TEST

ID: 234

Start: 2024-07-01

End: 2024-07-05

API Row: 0

filter by name

← https://eg278-it202-008-2026-1-qa.onrender.com/project/admin/list_golf.php?start_date=2025-01-01&end_date=2025-12-31

Home Profile Roles Golf Companies Logout

List Golf Tournaments

Golf Supremes

ID: 444

Start: 2025-05-01

End: 2025-05-05

API Row: 0

filter by date range

← https://eg278-it202-008-2026-1-qa.onrender.com/project/admin/list_golf.php?sort=name&order=ASC

Home Profile Roles Golf Companies Logout

List Golf Tournaments

Golf Supremes ID: 444 Start: 2025-05-01 End: 2025-05-05 API Row: 0	TEST ID: 234 Start: 2024-07-01 End: 2024-07-05 API Row: 0	The Sentry ID: 016 Start: 2024-01-04 End: 2024-01-07 API Row: 1	The Sentry ID: 016 Start: 2024-01-04 End: 2024-01-07 API Row: 1
---	--	--	--

sort by name ASC

← https://eg278-it202-008-2026-1-qa.onrender.com/project/admin/list_golf.php?name=XYZ123

Home Profile Roles Golf Companies Logout

List Golf Tournaments

No results to show

no results

← https://eg278-it202-008-2026-1-qa.onrender.com/project/admin/list_golf.php?limit=4

Home Profile Roles Golf Companies Logout

List Golf Tournaments

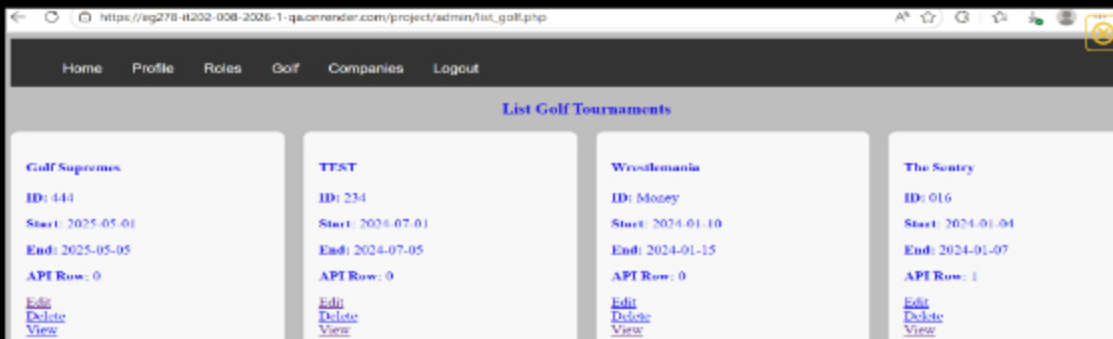
Golf Supremes ID: 444 Start: 2025-05-01 End: 2025-05-05	TEST ID: 234 Start: 2024-07-01 End: 2024-07-05	Wrestlemania ID: Money Start: 2024-01-10 End: 2024-01-15	The Sentry ID: 016 Start: 2024-01-04 End: 2024-01-07
---	--	--	--

URL: 2024-01-04	URL: 2024-01-04	URL: 2024-01-04	URL: 2024-01-04
API Row: 0	API Row: 0	API Row: 0	API Row: 1

limit=4 (4 tournaments displayed)



api row = 1



links(edit, view, delete)



Saved: 5/2/2026 6:14:46 PM

Task #3 (30 pts.) - Links

Progress: 100%

Details:

- Include the Render.com prod link to this page
- Include pull request link for this feature

URL #1

https://eg278-it202-008-prod.onrender.com/project/admin/list_golf.php



URL

https://eg278-it202-008-prod.onrender.com/project/admin/list_golf.php



URL #2

<https://github.com/eg278-cmd/eg278-it202-008/pull/552>



URL

<https://github.com/eg278-cmd/eg278-it202-008/pull/552>



<https://github.com/eg278-cmd/eg278-it202-008/pull/552>



Saved: 5/2/2026 6:23:27 PM

Section #4: (150 pts.) View Details Page

Section #4: (150 pts.) View Details Page (Single Item)

Progress: 100%

Task #1 (60 pts.) - Code

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code that handles loading of the record (and how the id retrieved for usage)
 - Include the code that handles invalid/missing ids (should redirect back to the previous page with an appropriate message)
- Show the code that generates the entity output
- Styling must be applied

```
// Retrieve ID  
$id = se($_GET, "id", -1, false);
```

where you retrieve the ID

```
// Validate ID BEFORE nav.php  
if ($id < 1) {  
    flash("Invalid tournament ID", "danger");  
    header("Location: " . get_url("admin/list_golf.php"));  
    exit;  
} <- #8-12 if ($id < 1)
```

Validation block

```
// Load record  
$query = "SELECT id, tourn_id, name, start_date, end_date, is_api  
FROM '11202-F24-golf'  
WHERE id = :id";  
$stmt = $dbh->prepare($query);  
  
try {  
    $stmt->execute(["id" => $id]);  
    $record = $stmt->fetch(PDO::FETCH_ASSOC);  
} catch (PDOException $e) {  
    error_log("Error loading tournament: " . var_export($e, true));  
    flash("Error loading tournament details", "danger");  
    header("Location: " . get_url("admin/list_golf.php"));  
    exit;  
} <- #16-21 catch (PDOException $e)  
  
// Handle missing record BEFORE nav.php  
if (!$record) {  
    flash("Tournament not found", "warning");  
    header("Location: " . get_url("admin/list_golf.php"));  
    exit;
```

loads the record from the database


```
<h3><?php se($record["name"]); ?></h3>
```

```
<p><strong>Tournament ID:</strong> <?php se($record["tourn_id"]); ?></p>
```

```
<p><strong>Start Date:</strong> <?php se($record["start_date"]); ?></p>
```

```
<p><strong>End Date:</strong> <?php se($record["end_date"]); ?></p>
```

```
<p><strong>API Row:</strong> <?php se($record["is_api"]); ?></p>
```

styling



view details page



Saved: 5/1/2026 6:28:50 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the loading logic works
- Briefly explain how the entity output is generated
- Briefly note the styling choices

Your Response:

- When it comes to the loading logic, the page will initially retrieve the tournament ID from the query string `$id = se($_GET, "id", -1, false)`; which will make sure the ID is extracted safely and defaults to -1 missing.
- The entity output is generated once the record is successfully loaded. It will show the tournament details inside of a container including each field (name, tournament ID, start date, end date, api row). You'll see a list of the main fields displaying the record's data alongside a "back to list" button.
- In regards to styling, the details page uses a container type box structure with light background colors, field names are bold and in blue color including the button.



Saved: 5/1/2026 6:28:50 PM

Task #2 (60 pts.) - Examples

Progress: 100%

Details:

- Show a few examples of different entities
- Each entity item should have more details than the summary view (if possible)
- Each entity should have the following links
 - A link to edit the single entity (this may be an admin-only thing, but it should be present for the respective role)
 - A link to delete the single entity (this may be an admin-only thing, but it should be present for the respective role)



examples section with edit/delete link



Saved: 5/1/2026 7:39:55 PM

Task #3 (30 pts.) - Links

Progress: 100%

Details:

- Include the Render.com prod link to this page
- Include pull request link for this feature

URL #1

https://eg278-it202-008-prod.onrender.com/project/admin/list_golf.php



URL

https://eg278-it202-008-prod.onrender.com/project/admin/list_golf.php



URL #2

<https://github.com/eg278-cmd/eg278-it202-008-2026/pull/56>



URL

<https://github.com/eg278-cmd/eg278-it202-008-2026/pull/56>



Saved: 5/1/2026 7:53:28 PM

Section #5: (150 pts.) Edit Page

Progress: 100%

Task #1 (60 pts.) - Code

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code that handles loading of the record (and how the id retrieved for usage)
 - Include the code that handles invalid/missing ids
- Show the code that generates the form with the correct data types and populates it
- Show the html, javascript, and php validation (should be similar to create)
- Styling must be applied

```
$id = se($_GET, "id", -1, false);

if ($id < 1) {
    flash("Invalid tournament ID", "danger");
    header("Location: " . get_url("admin/list_golf.php"));
    exit;
} <- #7-11 if ($id < 1)
```

the loading of the record and id being retrieved

```
if (!$golf) {
    flash("Tournament not found", "warning");
    header("Location: " . get_url("admin/list_golf.php"));
    exit;
} <- #31-35 if (!$golf)
```

missing ids

```
<label for="tourn_id">Tournament ID</label>
<input type="text" name="tourn_id" id="tourn_id" required
value "<?php se($golf, "tourn_id"); ?>"
class="mb-3">
<label for="name">Tournament Name</label>
<input type="text" name="name" id="name" required
value "<?php se($golf, "name"); ?>"
class="mb-3">
<label for="start_date">start date</label>
<input type="date" name="start_date" id="start_date" required
value "<?php se($golf, "start_date"); ?>"
class="mb-3">
<label for="end_date">End Date</label>
<input type="date" name="end_date" id="end_date" required
value "<?php se($golf, "end_date"); ?>"
```

form with correct data types

```
if ($<covered["request action"] == "secl") {
    // Only allow valid fields
    $allowed = ["tourn_id", "name", "start_date", "end_date"];
    $row = [];

    foreach ($allowed as $field) {
        if (<isset($_GET[$field])> {
            $row[$field] = se($_GET[$field], $field);
        }
    }
}
```

```

    $new['tourn_id'] = trim($_POST['tourn_id']);
    if ($id != 0) foreach ($allowed as $field) {
        $new[$field] = $id;
    }
    try {
        $result = update("TTOUR2 DB", $new);
        if ($result['rowCount'] > 0) {
            flash("Tournament updated successfully", "success");
        } else {
            flash("No changes detected", "warning");
        }
        header("Location: " . get_url("admin/edit-golf.php?id=" . $id));
        exit;
    } catch (PDOException $e) {
        error_log("Error updating tournament: " . var_export($e, true));
        flash("Error updating tournament", "danger");
    }
}

```

PHP Validation

```

<form method="POST">
  <div class="mb 4">
    <label for="tourn_id">Tournament ID</label>
    <input type="text" name="tourn_id" id="tourn_id" required
      value "<?php se($_GET, "tourn_id"); ?>" />
  </div>
  <div class="mb 3">
    <label for="name">Tournament Name</label>
    <input type="text" name="name" id="name" required
      value "<?php se($_GET, "name"); ?>" />
  </div>
  <div class="mb 4">
    <label for="start_date">Start Date</label>
    <input type="date" name="start_date" id="start_date" required
      value "<?php se($_GET, "start_date"); ?>" />
  </div>
  <div class="mb 3">
    <label for="end_date">End Date</label>
    <input type="date" name="end_date" id="end_date" required
      value "<?php se($_GET, "end_date"); ?>" />
  </div>
</form>

```

HTML Validation

```

<script>
function validateEditForm() {
    let name = document.getElementById("name").value.trim();

    if (name.length < 4) {
        alert("Name must be at least 4 characters");
        return false;
    }

    return true;
} <- #107-116 function validateEditForm()
</script>

```

JS Validation

 Saved: 5/1/2026 9:19:04 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the loading logic works
- Briefly explain how the form is generated and submission is handled
- Briefly note the styling choices
- Note what fields are editable and why the choice was made for them (it's likely not all fields are editable, especially id, created, modified, etc)

Your Response:

- Once the page loads, the script will then retrieve the id from the URL with the use of `se($_GET, "id", -1, false)`. However, if the ID was to be missing or invalid, the user will go back to the list page showing an error message. If it is valid, the tournament record will load successfully from the sql table. The output is fetched into an associative array(`$golf`).

Once no record found, the user will be redirected alongside a warning.

- The form uses the values of \$golf in order for the user to check and edit the existing data. When the form is submitted, the POST is running. The main fields like tourn_id, name, start_date, and end_date will only be excepted for execution. The record will be added to the update array and then flash messages will determine whether or not the update was successful.
- In regards to styling, elements of bootstrap are included container-fluid, mb-4, btn btn-primary to show spacing and the layout structure.
- The main fields like tourn_id, name, start_date, and end_date are able to be edited. id, created, modified are not editable due to the fact thhat they are just generated by the database. Users would cause conflicts with data if they were to modify or edit those fields



Saved: 5/1/2026 9:19:04 PM

Task #2 (60 pts.) - Examples

Progress: 100%

Details:

- Show a before and after of updating an entity (clearly caption)
- Successful update should have an appropriate message shown
- The updated data should be shown in the form
- Any errors should have user-friendly messages shown
- Show some examples of validation errors (html, javascript, php)



https://eg278-6202-008-prod.onrender.com/project/admin/edit_golf.php?id=20

Home Profile Roles Golf Companies Logout

Edit Golf Tournament

Tournament ID: 100

Tournament Name: Test Tournament

Start Date: 01/01/2023

End Date: 01/01/2024

Update

before updating entity(Test Tournament)



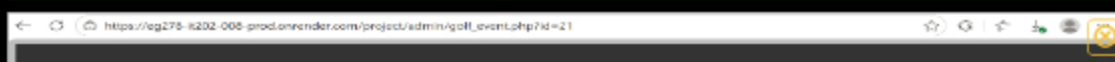
https://eg278-6202-008-prod.onrender.com/project/admin/list_golf.php

Home Profile Roles Golf Companies Logout

List Golf Tournaments

Golf Supremes ID: 444 Start: 2025-05-01 End: 2025-05-05 API Row: 0 Edit View	Test Tournament ID: TEST123 Start: 2025-01-01 End: 2025-01-05 API Row: 0 Edit View	Test Tournament ID: 100 Start: 2025-01-01 End: 2025-01-05 API Row: 0 Edit View	Bball ID: 234 Start: 2024-07-01 End: 2024-07-05 API Row: 0 Edit View
---	---	---	---

successful update



https://eg278-6202-008-prod.onrender.com/project/admin/golf_event.php?id=21

Home Profile Roles Golf Companies Logout

Edit Golf Tournament

Tournament ID: 100

Tournament Name: Test Tournament

Start Date: 01/01/2023

End Date: 01/01/2024

Update

Golf Tournament Details

Golf Supremes

Tournament ID: 444

Start Date: 2023-05-01

End Date: 2025-05-05

API Row: 0

[Back to List](#)

after updating entity(Golf Supremes)

HTML Validation Error

JS Validation Error



Saved: 5/1/2026 10:30:22 PM

Task #3 (30 pts.) - Links

Progress: 100%

Details:

- Include the Render.com prod link to this page
- Include pull request link for this feature

URL #1

https://eg278-it202-008-prod.onrender.com/project/admin/list_golf.php


URL

https://eg278-it202-008-prod.onrender.com/project/admin/list_golf.php


URL #2

<https://github.com/eg278-cmr/eg278-cmr>


URL

<https://github.com/eg278-cmr/eg278-cmr>




Saved: 5/1/2026 10:38:45 PM

Section #6: (150 pts.) Delete Logic

Progress: 100%

Task #1 (60 pts.) - Code

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code that handles deletion of the record (and how the id retrieved for usage)
 - Include the code that handles invalid/missing ids (should redirect to the list page with an appropriate message)
- Handle any necessary roles/permissions (i.e., it's not likely that anyone can delete any entity they choose)
- Show the redirect logic that goes back to the previous page (where the route came from), if it was a list page, it should maintain the filter/sort criteria

```

/** DELETE LOGIC */
$delete_id = se($_GET, "delete_id", -1, false);

if ($delete_id > 0) {
    // permission check
    if (!has_role("Admin")) {
        flash("You do not have permission to delete tournaments", "danger");
        header("Location: " . get_url("admin/list_golf.php"));
        exit;
    } <- #17-21 if (!has_role("Admin"))

    // Placeholder delete logic
    flash("Tournament deleted", "success");

    // Redirect back to the previous page
    $redirect = $_SERVER["HTTP_REFERER"] ?? get_url("admin/list_golf.php");
    header("Location: " . $redirect);
    exit;
}

```

deletion of the record and redirects to list page

```

// Permission check
if (!has_role("Admin")) {
    flash("You do not have permission to delete tournaments", "danger");
    header("Location: " . get_url("admin/list_golf.php"));
    exit;
}

```

handles any roles

```

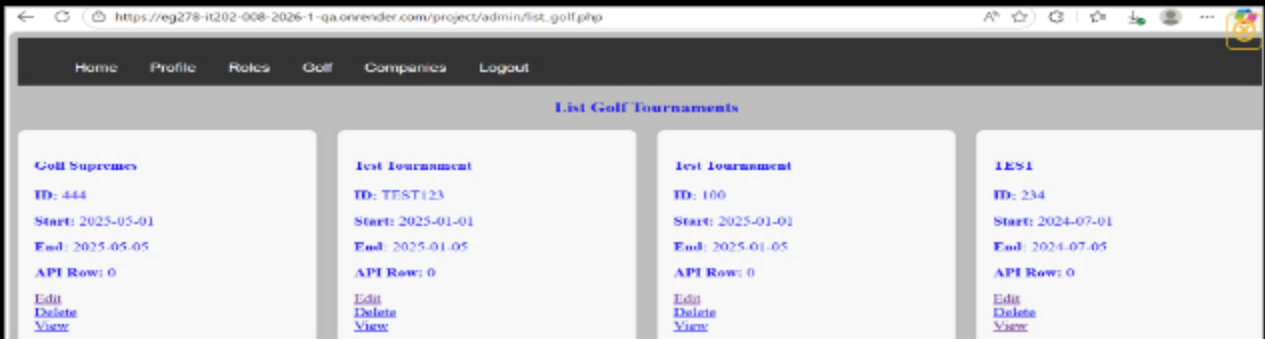
// Redirect back to the previous page
$redirect = $_SERVER["HTTP_REFERER"] ?? get_url("admin/list_golf.php");

```



```
header("Location: " . $redirect);  
exit;
```

redirect logic



Delete button

Saved: 5/2/2026 4:55:24 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the deletion logic works
- Is it a soft or hard delete?
- Note any permissions/rules applied.
- Briefly explain how the redirect maintains the previous route and any filter/sort data

Your Response:

- Anytime the user clicks the delete link, the page will send a request to list_golf.php? delete_id={id}. As long as the id is valid(> 0), the delete logic will successfully run. If by any chance the id is missing or invalid, then the condition fails which result in the page load regularly. Within the delete block, after the permissions are checked, a flash message will redirect the user back to the previous page. It would be a hard delete since the row is permanently deleted from the database and it no longer exist. Only users with an admin role can delete any tournaments. Non admin users will receive a permission

Saved: 5/2/2026 4:55:24 PM

Task #2 (60 pts.) - Examples

Progress: 100%

Details:

- Show a successful delete message

- Show a successful delete message
- Show a before and after database view of the record being deleted (clearly caption)
- Any errors should have user-friendly messages shown

successful delete message

before deleted

after deletion



Saved: 5/2/2026 4:52:30 PM

Task #3 (30 pts.) - Links

Progress: 100%

Details:

- Include the Render.com prod link to this page (even though it's not a full page, it's more like how logout works)
- Include pull request link for this feature

URL #1

https://eg278-it202-008-prod.onrender.com/project/admin/list_golf.php?delete_id=1



URL

<https://eg278-it202-008-prod.>



Saved: 5/2/2026 4:49:09 PM

Section #7: (50 pts.) Misc

Progress: 100%

Task #1 (16.67 pts.) - Github Details

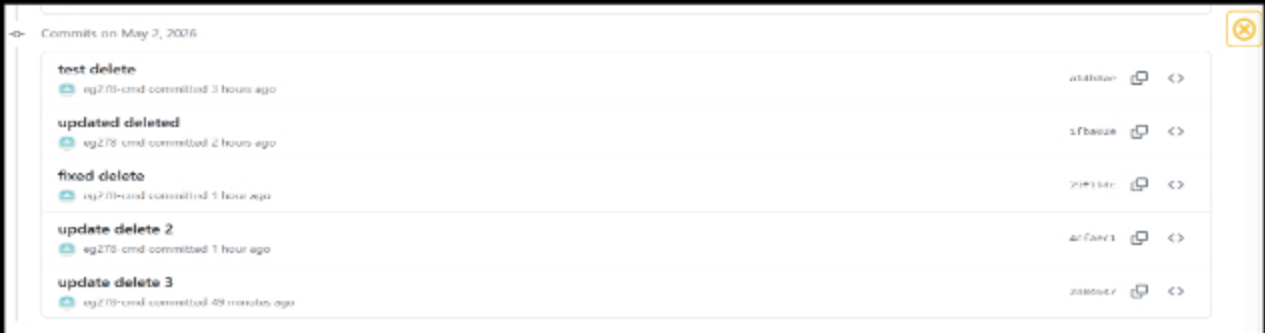
Progress: 100%

Part 1:

Progress: 100%

Details:

From the Commits tab of the Pull Request screenshot the commit history (it may not be possible to capture all, just grab a reasonable chunk)



The screenshot shows the 'Commits on May 2, 2026' tab for a Pull Request. It lists five commits with their messages, authors, and commit hashes. Each commit has a 'Details' link and a 'Compare' link.

Commit Message	Author	Commit Hash
test delete	eg278-cmd committed 3 hours ago	af14b0a
updated deleted	eg278-cmd committed 2 hours ago	1f8a028
fixed delete	eg278-cmd committed 1 hour ago	3f8154f
update delete 2	eg278-cmd committed 1 hour ago	4a7c0e1
update delete 3	eg278-cmd committed 49 minutes ago	23806a7

recent commit tab



Saved: 5/2/2026 4:46:46 PM

Part 2:

Progress: 100%

Details:

Include the link to the Pull Request (should end in /pull/#) (Milestone2 into qa)

URL #1

<https://github.com/eg278-cmd/eg278-it202-008-pull655>



URL

<https://github.com/eg278-cmd/eg>



Saved: 5/2/2026 4:46:46 PM

Task #2 (16.67 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click Projects and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary

Projects • eg278-it202-008-2026

21 hrs 15 mins over the Last 7 Days in eg278-it202-008-2026 under all branches. 📈

overall time at the top

Files		Branches	
6 hrs 16 mins	public_html/project/admin/list_golf.php	19 hrs 59 mins	qa
2 hrs 46 mins	partials/nav.php	1 hr 16 mins	Milestone2
2 hrs 27 mins	_ic_html/project/admin/create_golf.php		
2 hrs 19 mins	public_html/project/admin/golf_event.php		
1 hr 40 mins	lib/golf_api.php		
1 hr 15 mins	public_html/project/admin/edit_golf.php		
1 hr 4 mins	lib/.env		
1 hr	_project/sql/006_create_table_golf.sql		
32 mins	lib/config.php		

overall time at he bottom



Saved: 5/2/2026 4:45:43 PM

≡ Task #3 (16.67 pts.) - Reflection

Progress: 100%

⇒ Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

Based off of milestone 2, what i learned the most was how to correctly put my PHP, SQL, and HTML through code so i won't be able to run through any errors.



Saved: 5/2/2026 4:44:05 PM

⇒ Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part of the assignment was to create, fetch, and update my tournament record data.



Saved: 5/2/2026 4:41:22 PM

⇒ Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part of the assignment involved the delete logic but more so trying to manage the api integration alongside the admin pages.



Saved: 5/2/2026 4:39:32 PM